



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space: The set of all possible abstract physical configurations the environment can exist in (e.g., all unique (x, y) coordinate pairs available to the agent on the grid, bounded by grid size and walls). The size of the state space is finite and fixed by the environment layout.

Search Tree: An explicit graph/tree data structure generated dynamically during search. Nodes represent individual paths through the state space rooted at the start position. While the state space for an N times M grid contains N . M states, a search tree can have infinite nodes if loops are allowed because the same state can be reached via endless variations of paths.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

Problem Solved: It prevents repeated state expansions and infinite loops, converting a naive Tree Search into a **Graph Search**.

Impact on DFS: Without the `reached` set, DFS would fall into infinite loops (cycles) whenever the agent enters a two-node bidirectional hallway (e.g., oscillating endlessly back and forth between (0,0) and (0,1)).

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS Optimality: In a uniform-cost grid where every step costs 1, BFS expands nodes strictly level-by-level (shallowest depth first). Because edge costs are equal, the first time BFS reaches the target food node, it guarantees the shortest, optimal path.

DFS Suboptimality: DFS follows a single branch down to its maximum depth before backtracking. In a 2D grid, DFS blindly digs along a fixed move

precedence order (e.g., exploring Up until hit by a wall, then Down , Left , Right). It commits to the first path that hits the target regardless of total path length, generating winding, highly suboptimal routes.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

Search Strategy	Time Complexity	Space (Memory) Complexity	Memory Bottleneck Mechanics
BFS	$O(b^d)$	$O(b^d)$	Exponential Memory Limit. BFS must store every frontier node at the current level. On a massive grid, the memory grows exponentially with depth d , exhausting system RAM long before finding distant food.
DFS	$O(b^m)$	$O(b \cdot m)$	Linear Memory Efficiency. DFS only stores the active path from the root to the leaf node plus unexplored sibling nodes (where m is max depth). Memory demands are minimal ($O(m)$), but execution risks getting lost in deep branches.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

